5주차

게임 제작으로 배우는 체계적인 개발과 관리

할루시네이션 해결, 커스텀 명령어, 자동화 워크플로

신구대학교 컴퓨터소프트웨어학과 · 앱프로그래밍응용 (2026-1)

오늘 배울 내용

이 수업의 3대 핵심 역량

역량	설명
바이브 코딩	AI 코딩 파트너를 활용, 자연어 프롬프트로 빠르게 개발
컴퓨팅 사고	문제 분해 → 패턴 인식 → 추상화 → 알고리즘적 해결
프로젝트 관리	요구사항 → 일정 계획 → 중간 보고 → 최종 발표

오늘의 학습 목표

1. **할루시네이션**의 개념을 이해하고 교차 검증으로 AI 콘텐츠의 정확성을 높일 수 있다.
2. **커스텀 명령어**를 만들어 반복 작업을 자동화할 수 있다.
3. **인수와 플레이스홀더**를 활용해 커스텀 명령어를 확장할 수 있다.

> 📖 *교재: 혼자 공부하는 바이브 코딩 with 클로드 코드 — Chapter 05 (p.174~206)*

지난 주 복습

4주차에서 배운 것

- **단계별 프롬프트 전략** — PRD를 5단계 프롬프트로 분해하여 할 일 관리 앱 구현
- **세션 재개** — `claude --resume`, `--continue` 옵션으로 이전 작업 복원
- **컨텍스트 관리** — `/compact`로 압축, `/context`로 상태 확인
- **프로젝트 관리** — `/config`, `/doctor`, `/export` 명령어 활용

이번 주 핵심 질문

> 💬 *"AI가 만든 퀴즈 문제의 정답이 **정말 정확한지** 어떻게 검증할 수 있을까?"*

> 4주차까지 AI와 함께 코드를 작성하는 연습을 했습니다. 이번 주부터는 AI가 생성한 **콘텐츠의 정확성을 관리**하고, **반복 작업을 자동화**하는 방법을 배웁니다!

이번 주의 컴퓨팅 사고

컴퓨팅 사고의 **4가지 사고력**이 이번 주에 어떻게 적용되는지 살펴봅시다.

사고력	의미	이번 주 적용
분해	큰 문제를 작은 단위로 나누기	퀴즈 게임을 PRD → 3단계 프롬프트로 분해
패턴 인식	반복되는 구조 찾기	검증·문제추가·범위분석 등 반복 패턴을 커스텀 명령어로 추출
추상화	핵심만 뽑아 정리하기	교차 검증 가이드라인을 CLAUDE.md에 추상화
알고리즘	순서대로 절차 밟기	PRD→구현→테스트→검증→자동화 순서

💡 이번 주의 핵심은 **"패턴 인식"**! 반복되는 작업 패턴을 찾아 커스텀 명령어로 자동화하는 것이 바로 컴퓨팅 사고의 실전 적용입니다.

3대 역량, 이번 주 적용

역량	이번 주 활동
바이브 코딩	상식 퀴즈 게임 제작 + 커스텀 명령어로 반복 작업 자동화
컴퓨팅 사고	할루시네이션 감지 → 교차 검증 가이드라인 설계
프로젝트 관리	CLAUDE.md에 검증 규칙 문서화 + 커스텀 명령어로 품질 관리 체계 구축

바이브 코딩 = 퀴즈 게임 제작 + 커스텀 명령어 **자동화** (도구)
컴퓨팅 사고 = 할루시네이션을 **체계적으로 감지·검증** (사고)
프로젝트 관리 = 검증 가이드라인과 명령어를 **문서화·표준화** (계획)

할루시네이션이란? (교재 p.174)

할루시네이션(hallucination, 환각)은 생성형 AI가 사실과 다른 내용을 진짜처럼 만들어내는 현상입니다.

왜 발생할까?

- AI가 학습 데이터에서 **패턴을 찾아 이어 붙여** 확률적으로 가장 그럴듯한 문장을 생성하기 때문
- **불명확한 정보로 인한 추론** 오류

어떻게 대처할까?

- AI가 생성한 정보는 **신뢰할 수 있는 문서 또는 공식 사이트와 비교**해 교차 검증
- AI에게 **"출처 없는 정보는 생성하지 말 것"** 같은 규칙을 입력하여 사전 방지

> 💡 AI의 답변을 무조건 신뢰하지 마세요! 특히 **사실 기반 콘텐츠**(퀴즈, 역사, 과학)는 반드시 교차 검증이 필요합니다.

상식 퀴즈 게임 PRD 작성하기 (교재 p.175~176)

클로드 웹에서 PRD 작성

```
상식 퀴즈 게임을 만들려고 해. PRD를 작성해 줘.

게임 규칙:
- 4지선다 객관식 퀴즈
- 카테고리: 한국사, 과학, 지리, 일반상식
- 각 카테고리별 10문제씩 총 40문제
- 정답/오답 즉시 피드백
- 최종 점수 및 순위 기록
```

PRD → 3단계 프롬프트 변환

```
상식 퀴즈 게임을 클로드 코드로 구현할 수 있도록 PRD를
바탕으로 3단계 프롬프트를 정리해 줘.
```

💡 2장에서 배운 5W1H 원칙을 떠올리며 요구사항을 정리해 보세요.

퀴즈 게임 구현하기 (교재 p.177~178)

프로젝트 준비

1. `VibeCoding` 폴더에 `Study-03` 폴더 생성
2. 명령 프롬프트에서 `claude` 실행
3. `/init` 입력하여 CLAUDE.md 자동 생성

1단계 프롬프트 입력

클로드 웹에서 만든 1단계 프롬프트를 **복사하여 붙여넣기**합니다.

구현 결과

- `CLAUDE.md`, `index.html`, `quiz-data.js`, `quiz-game.js` 등 파일 생성
- Vanilla JS로 구현된 반응형 웹 애플리케이션
- 문제 데이터 구조: 4개 카테고리 × 10문제 = 총 40문제
- 난이도별 점수 시스템 (쉬움:10점, 보통:15점, 어려움:20점)

> 📖 *교재 소스코드: https://github.com/taehojo/vibecoding (Study-03)*

게임 테스트 (교재 p.179~180)

테스트 방법

1. `Study-03` 폴더에서 `index.html` 파일을 **더블 클릭**하여 브라우저에서 실행
2. [게임 시작] 버튼 클릭 후 문제 풀기
3. 정답/오답 즉시 피드백 제공 확인
4. 모든 퀴즈를 풀면 최종 점수 표시

확인 항목

항목	확인 내용
카테고리	4개 카테고리 문제가 순차적으로 출제되는가
피드백	정답 → ✅ "정답입니다!", 오답 → ❌ "틀렸습니다" + 해설
점수	최종 점수, 정답 개수, 정답률 표시
카테고리별 결과	카테고리별 정답 수(예: 한국사 7/10)

할루시네이션 해결하기 (교재 p.180~181)

할루시네이션 사례: "세계에서 가장 긴 강"

- 퀴즈에서 "세계에서 가장 긴 강"의 정답으로 **아마존강**을 선택했지만 → **틀림!** 정답은 나일강
- 그런데 최근 연구에서 아마존강의 수원지를 재측정한 결과, **아마존강이 더 길다는 주장도 제기**
- 이처럼 **논란의 여지가 있는 주제**는 교차 검증을 거쳐 명확한 기준을 세워야 함

대응 전략

- 논란 있는 정보의 검증을 위해 **검증 가이드라인**을 마련
- CLAUDE.md 파일에 영구적으로 저장하는 것이 가장 효과적

> **컴퓨팅 사고 연결**: 애매한 표현("가장 긴", "최초의")을 발견하고 **명확한 기준**(전통적 측정 기준)을 추가하는 것이 바로 **추상화** 사고력입니다.

교차 검증 가이드라인 구축하기 (교재 p.182~185)

/memory 명령어로 CLAUDE.md 편집

```
/memory
```

- 메모리 저장 위치 선택:
  - **1. Project memory** — 현재 프로젝트 폴더 안에서만 적용 (→ `./CLAUDE.md`)
  - **2. User memory** — 모든 프로젝트에 공통 적용 (→ `~/.claude/CLAUDE.md`)
- 퀴즈 가이드라인은 현재 프로젝트 전용이므로 **1번** 선택

교차 검증 가이드라인 추가 내용 (교재 p.182~185)

CLAUDE.md에 아래 가이드라인을 추가합니다:

```
퀴즈 문제 교차 검증 가이드라인

모든 문제 작성 시 확인 사항

1. 정답이 하나뿐인가?
   - 다른 해석 가능 시 조건 명시 (예: 면적 기준, 2024년 기준)
2. 최상급 표현에 기준이 있는가?
   - '가장 큰', '최초의' 등 표현에 측정 기준 명시
3. 시간과 범위가 명확한가?
   - 변할 수 있는 정보는 시점 명시
   - 지리적, 분류적 범위 한정
4. 교차 검증했는가?
   - 의심스러운 정보는 2개 이상 출처 확인
   - 논란 있는 내용은 주류 학설 기준
```

/memory와 CLAUDE.md (교재 p.183~185)

/memory 실행 결과

```
> /memory
L Opened memory file at ./CLAUDE.md

> To use a different editor, set the $EDITOR
  or $VISUAL environment variable.
```

CLAUDE.md 저장 후 확인

가이드라인을 추가한 뒤 **[파일]→[저장(Ctrl+S)]** 메뉴로 저장하고, 에디터를 닫습니다.

프롬프트로 확인

```
현재 프로젝트의 CLAUDE.md 내용을 확인해 줘.
```

💡 CLAUDE.md에는 형식이 정해져 있지 않으므로, **읽기 쉬운 자연어로 간결하게** 작성하세요. 중요한 규칙은 **'중요', '반드시'** 같은 강조 표현을 추가하면 AI가 더 잘 따릅니다. (교재 p.184)

/clear와 /compact의 차이 (교재 p.186)

/clear — 컨텍스트 초기화

```
/clear
```

현재 세션의 대화와 작업 컨텍스트를 모두 지우고 **완전히 초기화**합니다.

/clear vs /compact 비교

구분	/clear	/compact
기본 동작	대화 기록을 완전히 초기화	대화 내용을 요약하여 압축
컨텍스트 윈도우	완전히 비움	요약된 형태로 유지
CLAUDE.md 파일	유지됨	유지됨
사용 시점	새 주제 전환, 잘못된 방향 리셋	같은 작업 계속, 토큰 절약

가이드라인 점검하기 (교재 p.187~189)

/clear 후 가이드라인 적용 테스트

```
/clear
```

`/clear`로 대화를 초기화한 뒤, CLAUDE.md에 저장된 가이드라인대로 작동하는지 테스트합니다.

프롬프트

```
지금까지 만든 문제를 방금 저장한 가이드라인을 참고해서
점검해 줘. 가이드라인에 맞지 않은 퀴즈의 판단이 있다면
이에 맞게 수정해 줘.
```

검증 결과 예시 (교재 p.187)

- "세계에서 가장 큰 대륙은?" → "세계에서 **면적이** 가장 큰 대륙은?"
  - 측정 기준(면적)을 명시하여 답의 명확성 확보
- "세계에서 가장 높은 산은?" → "세계에서 **해발고도가** 가장 높은 산은?"
  - 측정 기준(해발고도)을 명시하고 정확한 수치(8,848m) 추가

✏️ 셀프 체크 — 05-1 할루시네이션과 검증

(교재 p.190)

문제 1. 다음 설명 중 틀린 것을 고르세요.

① AI 할루시네이션은 패턴을 과도하게 일반화할 때 발생할 수 있다.

② /memory 명령어에서 Project memory를 선택하면 ./CLAUDE.md 파일이 열린다.

③ /clear는 대화를 요약하여 압축하고 컨텍스트를 유지한다.

④ ./CLAUDE.md에 가이드라인을 추가할 때는 자연어로 간결하고 명확하게 작성하면 된다.

✏️ 셀프 체크 — 05-1 (계속)

문제 2. 다음 상황에서 사용할 적절한 명령어나 대응 방법을 쓰세요.

상황	대응
퀴즈 게임의 검증 가이드라인을 저장하고 싶을 때	
이전 대화가 새 작업에 영향을 줄 것 같을 때	
대화가 길어져서 토큰을 절약하고 싶을 때	
"세계에서 가장 긴 강" 같은 애매한 문제를 발견했을 때	

커스텀 명령어란? (교재 p.191, p.194)

- **커스텀 명령어**(custom command)란 사용자가 직접 정의해서 쓸 수 있는 **나만의 단축 지시어**입니다.

왜 커스텀 명령어가 필요한가?

- 문제 형식 통일, 가이드라인 검증, 난이도 균형 확인 등 **반복 작업이 많음**
- 매번 긴 프롬프트를 입력하면 효율이 떨어짐
- 명령어는 **마크다운(markdown) 파일**로 정의되며, 범위에 따라 2가지로 나눌

커스텀 명령어의 두 종류

종류	저장 위치	범위
전역 명령어	`~/.claude/commands/`	모든 프로젝트에서 사용 가능
프로젝트 명령어	`.claude/commands/`	해당 프로젝트에서만 사용 가능

2단계: 게임 확장하기 (교재 p.192~194)

2단계 프롬프트 실행 결과

기능	설명
점수 시스템	난이도별 배수(easy 1x / medium 1.5x / hard 2x) + 시간·힌트·콤보 보너스
전체 도전	40문제 전체 풀기
카테고리별	선택한 카테고리 10문제
스피드 퀴즈	20문제, 문제당 15초 제한

명령어 폴더 생성하기 (교재 p.195)

프롬프트

```
현재 프로젝트에 커스텀 명령어 폴더를 만들어 줘.
.claude/commands 디렉터리를 생성하고 구조를 보여줘.
```

생성된 디렉터리 구조

```
Study-03\
 ├─── .claude/
 │    └─── commands/        ← 커스텀 명령어 저장 폴더
 ├─── index.html
 ├─── styles.css
 ├─── quiz-data.js
 ├─── quiz-game.js
 └─── CLAUDE.md
```

> `.claude/commands` 폴더는 Claude Code에서 커스텀 슬래시 명령어를 정의하는 데 사용됩니다. 이 폴더에 마크다운 파일을 추가하면 프로젝트별 맞춤 명령어를 만들 수 있습니다.

첫 번째 커스텀 명령어 만들기 (교재 p.195~197)

/quiz-validate 명령어 생성

```
.claude/commands/quiz-validate.md 파일을 만들어 줘.
퀴즈 문제 중에서 '가장', '최초', '최대' 같은 최상급 표현이
있는지 찾아서 목록으로 보여줘.
```

명령어 실행 방법

1. `/exit` 으로 클로드 코드 **종료**
2. `claude` 명령어로 **재시작**
3. `/(슬래시)` 입력 시 `/quiz-validate` 명령어 확인
4. `/quiz-validate` 실행

실행 결과

최상급 표현이 포함된 문제들을 찾아내고, 측정 기준이 명확한지 점검해 줍니다.

> 💡 `/help` 명령어를 입력하면 등록된 모든 명령어를 볼 수 있습니다. (교재 p.197)

커스텀 명령어 확장하기 — 인수와 플레이스홀더 (교재 p.198)

인수(argument)는 명령어나 함수에 전달하는 추가 정보로, 명령어가 '무엇을 해야 할지'를 정한다면, 인수는 '무엇을 대상으로 어떻게 할 것인가'를 구체적으로 알려줍니다.

플레이스홀더(placeholder)란?

- `$ARGUMENTS`, `$1`, `$2` 처럼 인수가 들어갈 자리를 표시하는 기호
- 실행 시 사용자가 입력한 인수로 **자동 치환**

두 가지 확장 방식

```
/quiz-validate 한국사        ← 특정 카테고리만 검증
/quiz-range 1 20             ← 1~20번 문제만 분석
```

💡 플레이스홀더는 나중에 실제 값이 들어갈 자리를 미리 만들어 놓는 기호입니다. 마치 "환영합니다, 혼공바이브님"에서 사용자 이름이 폴더명 플레이스홀더로 대체되는 것과 같습니다. (교재 p.198)

특정 카테고리만 검증하기 (교재 p.199~200)

quiz-validate.md 인수 추가

```
.claude/commands/quiz-validate.md를 다음과 같이 수정해 줘.

사용자가 카테고리를 지정하면 해당 카테고리의 문제만 검증하고,
지정하지 않으면 전체 문제를 검증해 줘.
$ARGUMENTS에 입력된 값을 카테고리로 사용하면 돼.

검증할 때는 '가장', '최초', '최대' 같은 모호한 표현이 있는지
찾아서 어떤 기준을 명시해야 하는지 알려 줘.
```

실행 예시

```
/quiz-validate 한국사
```

- `한국사` 카테고리의 퀴즈만 검증
- 최상급 표현 사용 문제, 측정 기준 명확성, 개선 필요 문제 등을 보고

특정 범위만 분석하기 (교재 p.200~201)

/quiz-range 명령어 생성

```
.claude/commands/quiz-range.md를 생성해.

$1번부터 $2번까지의 문제를 검토하는 기능을 만들어 줘.
문제의 난이도와 정답 분포를 확인하게 해 줘.
```

실행 예시

```
/quiz-range 1 20
```

분석 내용

- 1~20번 문제의 **난이도 분포** 확인
- **정답 위치 편향** 분석 (예: 정답이 0번에 55% 집중)
- 3번 선택지 미사용, 정답이 1번에만 집중 등 **패턴 감지**
- 정답 위치를 균등하게 분산시키도록 **권장 사항** 제시

새로운 퀴즈 문제 추가 자동화 (교재 p.202~204)

/quiz-add 명령어 생성

```
.claude/commands/quiz-add.md를 생성해.

새 퀴즈를 추가하는 명령어를 만들어 줘.
$1은 카테고리, $2는 난이도로 받아서 처리해.
기존 퀴즈들과 같은 형식으로 만들고, 검증 가이드라인을 꼭
지키게 해 줘.
```

실행 예시

```
/quiz-add 한국사 medium
```

실행 결과

- `quiz-data.js`에 새로운 한국사 문제(ID: 41) 추가
- 카테고리별 정답 위치 편향 **자동 개선**
- 전체 문제 수: 40개 → 41개

✏️ 셀프 체크 — 05-2 커스텀 명령어와 자동화

(교재 p.205)

문제 1. 다음 설명 중 틀린 것을 고르세요.

① 프로젝트 명령어는 해당 프로젝트에서만 사용 가능하다.
② 커스텀 명령어는 자연어로 작성할 수 있다.
③ 명령어 생성 후 즉시 사용 가능하다.
④ 마크다운 파일 형식으로 명령어를 저장한다.
⑤ 매개변수를 통해 유연한 동작이 가능하다.

✏️ 셀프 체크 — 05-2 (계속)

(교재 p.206)

문제 2. 다음 빈칸에 들어갈 적절한 답을 쓰세요.

- 프로젝트 전용 커스텀 명령어가 저장되어야 할 위치는 ( )이다.
- 새 명령어 작성 후 사용가능하게 하려면 클로드 코드를 ( )해야 한다.
- 커스텀 명령어에서 특정 번호 범위의 문제만 검토하려면 (,)와 같이 인수 플레이스 홀더를 사용한다.
- 커스텀 명령어에서 사용자 입력값을 받으려면 ( )와 같은 인수 플레이스 홀더를 사용한다.
- `~/.claude/commands`에 만드는 명령어는 ( )에서 사용 가능하다.
- `.claude/commands`에 만드는 명령어는 ( )에서 사용 가능하다.

실습을 시작합니다!

지금부터 **상식 퀴즈 게임**을 만들고, **할루시네이션을 해결**하며, **커스텀 명령어**로 자동화합니다.

실습 흐름

```
① PRD 작성 → ② 퀴즈 게임 구현 → ③ 게임 테스트 →
④ 교차 검증 가이드라인 → ⑤ 커스텀 명령어 만들기 → ⑥ 명령어 확장
```

준비물

- 인터넷 연결된 PC (Windows)
- 명령 프롬프트 (cmd) 또는 터미널
- Claude 계정 (https://claude.ai)
- `VibeCoding/Study-03` 폴더 생성

실습 ① PRD 작성 & 퀴즈 게임 구현

1단계: 클로드 웹에서 PRD 작성

```
상식 퀴즈 게임을 만들려고 해. PRD를 작성해 줘.

게임 규칙:
- 4지선다 객관식 퀴즈
- 카테고리: 한국사, 과학, 지리, 일반상식
- 각 카테고리별 10문제씩 총 40문제
- 정답/오답 즉시 피드백
- 최종 점수 및 순위 기록
```

2단계: PRD → 3단계 프롬프트 변환

```
상식 퀴즈 게임을 클로드 코드로 구현할 수 있도록 PRD를
바탕으로 3단계 프롬프트를 정리해 줘.
```

실습 ① 퀴즈 게임 구현 (계속)

3단계: 클로드 코드에서 1단계 프롬프트 실행

```
mkdir Study-03
cd Study-03
claude
```

```
/init
```

- 변환된 1단계 프롬프트를 **복사 → 붙여넣기**
- 완료 메시지 확인 후 `index.html`을 브라우저에서 실행

[확인] 체크리스트

- [ ] 4개 카테고리의 문제가 순차적으로 출제되는가?
- [ ] 정답/오답 즉시 피드백이 제공되는가?
- [ ] 최종 점수와 카테고리별 결과가 표시되는가?

실습 ② 할루시네이션 검증 & 가이드라인 작성

/memory로 CLAUDE.md 편집

1. `/memory` → **1. Project memory** 선택
2. CLAUDE.md 맨 아래에 **교차 검증 가이드라인** 추가 → `Ctrl+S` 저장

/clear 후 가이드라인 테스트

```
/clear
```

```
지금까지 만든 문제를 방금 저장한 가이드라인을 참고해서
점검해 줘. 가이드라인에 맞지 않은 퀴즈의 판단이 있다면
이에 맞게 수정해 줘.
```

[확인] 체크리스트

- [ ] 기존 퀴즈 중 최상급 표현 문제가 수정되었는가?
- [ ] "전통적 측정 기준" 같은 조건이 문제에 추가되었는가?

실습 ③ 커스텀 명령어 만들기 — /quiz-validate

명령어 폴더 생성 & /quiz-validate 생성

```
현재 프로젝트에 커스텀 명령어 폴더를 만들어 줘.
.claude/commands 디렉터리를 생성하고 구조를 보여줘.
```

```
.claude/commands/quiz-validate.md 파일을 만들어 줘.
퀴즈 문제 중에서 '가장', '최초', '최대' 같은 최상급 표현이
있는지 찾아서 목록으로 보여줘.
```

실행: `/exit` → `claude` → `/quiz-validate`

실습 ④ 커스텀 명령어 확장 — 인수 추가

/quiz-validate에 카테고리 인수 추가

```
.claude/commands/quiz-validate.md를 다음과 같이 수정해 줘.
$ARGUMENTS에 입력된 값을 카테고리로 사용하면 돼.
```

/quiz-range 명령어 생성

```
.claude/commands/quiz-range.md를 생성해.
$1번부터 $2번까지의 문제를 검토하는 기능을 만들어 줘.
```

[확인] → `/quiz-validate 한국사` · `/quiz-range 1 20` 실행

실습 ⑤ 퀴즈 추가 자동화 — /quiz-add

/quiz-add 명령어 생성

```
.claude/commands/quiz-add.md를 생성해.
새 퀴즈를 추가하는 명령어를 만들어 줘.
$1은 카테고리, $2는 난이도로 받아서 처리해.
검증 가이드라인을 꼭 지키게 해 줘.
```

실행: `/exit` → `claude` → `/quiz-add 한국사 medium`

[확인] 체크리스트

- [ ] 새로운 퀴즈가 `quiz-data.js`에 추가되었는가?
- [ ] 추가된 퀴즈가 기존 형식과 일관성을 유지하는가?

실습 중 자주 발생하는 문제

문제	원인	해결 방법
커스텀 명령어가 목록에 안 나옴	클로드 코드 재시작 안 함	`/exit` 후 `claude`로 재시작
`/quiz-validate` 실행 오류	.md 파일 경로 오류	`.claude/commands/` 폴더 확인
프롬프트가 너무 길어 요약 표시됨	긴 텍스트 붙여넣기	`[Pasted text #1 +44 lines]` 메시지가 뜨면 `Enter`로 진행
`/memory`에서 에디터 안 열림	기본 에디터 미설정	메모장으로 CLAUDE.md 직접 편집
`/clear` 후 이전 작업 사라짐	/clear는 완전 초기화	이어서 하려면 `/compact` 사용

핵심 키워드 — 이론 (교재 p.190)

> 📖 *3가지 키워드로 끝내는 핵심 포인트*

- **할루시네이션**(hallucination) — AI가 그럴듯하지만 사실과 다른 정보를 진짜처럼 만들어내는 현상
- **/memory** — AI가 기억하는 프로젝트 관련 정보를 확인하고 관리하는 명령어
- **/clear** — 현재 세션의 대화와 작업 맥락을 모두 지우고 초기화하는 명령어

핵심 키워드 — 실습 (교재 p.205)

> 📖 *5가지 키워드로 끝내는 핵심 포인트*

- **커스텀 명령어**(custom command) — 사용자가 직접 정의해 반복 작업을 자동화할 수 있는 개인화된 명령어
- **전역 명령어** — 홈 디렉터리(`~/.claude/commands/`)에 정의되어 모든 프로젝트에서 공통으로 사용
- **프로젝트 명령어** — 현재 작업 폴더(`.claude/commands/`)에 저장되어 해당 프로젝트에서만 적용
- **인수**(argument) — 명령어나 함수에 전달되는 추가 정보로, 작업의 대상이나 범위를 지정하는 역할
- **플레이스홀더**(placeholder) — `$ARGUMENTS`, `$1`, `$2`처럼 인수가 들어갈 자리를 표시하는 기호로, 실행 시 실제 값으로 치환

오늘의 마무리 체크리스트

수업이 끝나기 전에 아래 항목을 확인해 보세요!

- [ ] 클로드 웹에서 **상식 퀴즈 게임 PRD를 작성**하고 3단계 프롬프트로 변환 완료
- [ ] 클로드 코드에서 **퀴즈 게임을 구현**하고 테스트 완료
- [ ] **할루시네이션 사례**를 인식하고 교차 검증의 필요성 이해
- [ ] `/memory`로 **CLAUDE.md에 교차 검증 가이드라인** 저장
- [ ] `/clear`와 `/compact`의 **차이점** 이해
- [ ] **커스텀 명령어** `/quiz-validate`, `/quiz-range`, `/quiz-add` 만들기
- [ ] "내가 오늘 배운 것" 2줄 메모

✅ 퀴즈 게임 실행 화면 캡처 + 커스텀 명령어 실행 결과 캡처를 교수님께 제출하면 오늘 수업 완료!

최신 업데이트 — 커스텀 명령어의 진화 (2026)

> 📢 *교재 출간 이후 Claude Code가 크게 업데이트되었습니다!*

커스텀 명령어 → 스킬(Skills) 진화

구분	기존 (교재)	최신
저장 위치	`.claude/commands/`	`.claude/skills/<이름>/SKILL.md`
파일 형식	마크다운	마크다운 + **YAML frontmatter**
호출 제어	수동(`/명령어`)만 가능	수동 + **Claude 자동 호출** 가능
호환성	—	기존 commands 파일 **그대로 작동**

Auto Memory — Claude가 스스로 기억

- `/memory` 실행 시 **auto memory 토글** 가능 (v2.1.59+)
- Claude가 빌드 명령어, 디버깅 인사이트 등을 **자동 저장**
- 저장 위치: `~/.claude/projects/<프로젝트>/memory/`

💡 기존 `.claude/commands/`는 그대로 호환됩니다. 수업에서는 교재대로 commands를 사용하세요!

최신 업데이트 — 새로운 기능들 (2026)

CLAUDE.md 정리 기능 확장

- `.claude/rules/` 디렉터리로 규칙을 주제별 파일로 분리 관리
- `@파일경로` import 문법으로 CLAUDE.md에서 다른 파일 참조 가능

새로운 내장 명령어

명령어	기능
`/plan`	계획 모드로 진입하여 작업 설계
`/diff`	변경 사항을 인터랙티브하게 확인
`/rewind`	대화/코드를 이전 시점으로 되돌리기
`/skills`	등록된 스킬 목록 보기

기타 업데이트

- 플레이스홀더: `$ARGUMENTS[0]`, `$0` — **0-based 인덱스** 단축 문법
- `/clear`의 새 별칭: `/reset`, `/new`

과제 안내

이번 주 과제

오늘 만든 상식 퀴즈 게임을 2단계 프롬프트로 확장하고, `/quiz-add` 명령어를 활용해 새 문제를 추가하세요.

과제 요구사항

- `/quiz-add` 명령어로 새 문제 추가 (검증 가이드라인 준수)
- `/quiz-validate`로 추가 문제 **검증 결과** 캡처
- `/quiz-range`로 전체 문제 **난이도 분포 분석** 캡처
- 실행 결과 **스크린샷** 제출

다음 주 예고

> ***6주차**: 클로드 코드에 API로 날개 달기 — 외부 서비스 연동과 API 활용*

> 오늘 배운 "할루시네이션 대응"과 "커스텀 명령어 자동화"는 앞으로 AI를 활용한 **모든 프로젝트에서 품질과 효율을 동시에 높이는** 핵심 기법입니다!